KUBERNETES

1 DE OUTUBRO DE 2024

# DESCOMPLICANDO GATEWAY API NO KUBERNETES

Neste artigo completo da LINUXtips, escrito por **Jeferson Fernando**, você aprenderá a montar um cluster Kubernetes utilizando o **KIND** (Kubernetes IN Docker) e a configurar o **NGINX Gateway Fabric** para gerenciar eficientemente o tráfego das suas aplicações. Abordaremos desde a instalação dos pré-requisitos essenciais, como **Docker**, **kubectl** e **KIND**, até configurações avançadas de roteamento utilizando **HTTPRoute**, incluindo balanceamento de carga, redirecionamentos, rate limiting e segurança **TLS**.

Além disso, você descobrirá como implantar aplicações, testar o roteamento, escalar serviços e monitorar logs para garantir um ambiente Kubernetes robusto, seguro e escalável. Este artigo da LINUXtips.io é ideal para desenvolvedores e administradores que desejam otimizar a gestão de tráfego em seus clusters Kubernetes de maneira eficiente e segura.

Prepare-se para transformar sua infraestrutura com soluções avançadas e seguras!

## Pré-requisitos

Antes de começarmos, certifique-se de que sua máquina possui os seguintes componentes **instalados** e **funcionando corretamente**:

- **Docker**: Fundamental para a criação de contêineres. Instale o Docker.
- **kubectl**: Ferramenta de linha de comando para interagir com Kubernetes. Instale o kubectl.
- **KIND**: Utilizado para criar clusters Kubernetes dentro de contêineres Docker. Instale o KIND.

> **Dica:** Caso ainda não tenha algum desses componentes, recomendo conferir os tutoriais oficiais ou os vídeos da **LINUXtips** para uma instalação detalhada.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# O que é a Gateway API e por que ela é melhor que o Ingress no Kubernetes?

A **Gateway API** é uma iniciativa mais recente no Kubernetes, projetada para superar as limitações do **Ingress**, oferecendo um conjunto de APIs mais **flexíveis** e **extensíveis** para gerenciar o tráfego de rede. Ela não substitui o Ingress, mas evolui a partir dele, trazendo soluções para demandas mais complexas e necessidades não atendidas pelo modelo de Ingress.

## EVOLUÇÃO DO INGRESS

O Ingress é amplamente utilizado e, para muitas aplicações, cumpre bem seu papel. No entanto, conforme as infraestruturas crescem e as demandas por funcionalidades avançadas aumentam, o Ingress começa a mostrar suas limitações. A **Gateway API** surgiu justamente para atender a esses cenários mais sofisticados, oferecendo uma abordagem modular, separação clara de responsabilidades e suporte a diversos protocolos, como HTTP, HTTPS, TCP e TLS, de forma nativa.

## COMO FUNCIONA A GATEWAY API?

A **Gateway API** introduz uma estrutura hierárquica de recursos, como **GatewayClass**, **Gateway**, **HTTPRoute**, **TLSRoute** e **TCPRoute**, permitindo uma separação mais clara das responsabilidades e maior flexibilidade na configuração do tráfego de rede. Veja alguns dos principais benefícios:

- **Extensibilidade:** Permite que os provedores de infraestrutura adicionem funcionalidades específicas sem comprometer a compatibilidade com a API padrão.
- **Controle Granular:** Suporta cenários complexos, onde diferentes equipes podem gerenciar diferentes aspectos do tráfego de rede, como roteamento e segurança.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Benefícios da Gateway API em Relação ao Ingress

Abaixo, destacamos os principais diferenciais da Gateway API em relação ao Ingress:

## Flexibilidade e Extensibilidade

- **Ingress:** Limitado a roteamento de HTTP/HTTPS e sem suporte nativo a outros protocolos.
- **Gateway API:** Suporta múltiplos protocolos (HTTP, HTTPS, TCP, TLS) e permite a adição de novas funcionalidades sem modificar a API principal, proporcionando maior flexibilidade para diferentes cenários.

## Controle e Granularidade

- **Ingress:** Possui uma API mais simples e direta, ideal para cenários menos complexos, mas não oferece o mesmo nível de controle granular sobre o roteamento e balanceamento de carga.
- **Gateway API:** Oferece maior controle sobre o comportamento do tráfego, permitindo definir regras avançadas de roteamento, como baseado em cabeçalhos, políticas de segurança e balanceamento de carga por diferentes protocolos.

## Separação de Preocupações

- **Ingress:** Combina a definição de regras de roteamento com a implementação do controlador de ingressos, limitando a capacidade de gerenciamento por diferentes equipes.
- **Gateway API:** A separação entre os recursos **Gateway** (gerencia o ponto de entrada) e **HTTPRoute** ou **TCPRoute** (definem as regras de roteamento) permite uma gestão mais organizada e distribuída, facilitando a colaboração entre equipes de infraestrutura e desenvolvimento.

## Suporte a Funcionalidades Avançadas

- **Ingress:** Funcionalidades avançadas, como roteamento por cabeçalhos ou autenticação, dependem fortemente do controlador específico de Ingress, sem padronização.
- **Gateway API:** Oferece suporte nativo a recursos avançados de roteamento, permitindo uma padronização consistente entre diferentes implementações e provedores de serviços.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Quando Usar o Ingress e Quando Usar a Gateway API?

- **Ingress:** Ideal para cenários onde uma solução simples de roteamento HTTP/HTTPS é suficiente. Se você já utiliza o Ingress no seu cluster e ele atende às suas necessidades atuais, pode não haver necessidade imediata de migrar.
- **Gateway API:** Recomendado para infraestruturas mais complexas, onde há a necessidade de suporte a protocolos adicionais como **TCP** ou **TLS**, maior flexibilidade de **customização** e **controle de tráfego**, ou quando diferentes equipes precisam gerenciar partes distintas da configuração de rede.

> **Dica do Jefim:** Avalie bem as necessidades do seu projeto antes de decidir entre **Ingress** e **Gateway API**. Se a flexibilidade, suporte a múltiplos protocolos e o controle granular forem fundamentais, a **Gateway API** é o caminho ideal!

# Representação Avançada do Ingress

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
                +----------------------------------------+
                | Requisições Externas (HTTP/HTTPS)      |
                |        (Usuários / Aplicações)         |
                +----------------------------------------+
                                |
                                v
                 +------------------------------------+
                 |     Controlador de Ingress         |
                 | (Trata das regras de roteamento)   |
                 +------------------------------------+
                                |
          +-------------------+-------------------+
          |                                       |
          v                                       v
     +----------------------+                +----------------------+
     | Regra de Ingress 1   |                | Regra de Ingress 2   |
     | (/app1 -> Serviço A)|                | (/app2 -> Serviço B)|
     +----------------------+                +----------------------+
          |                                       |
          v                                       v
     +----------------+                     +----------------+
     |   Serviço A    |                     |   Serviço B    |
     |   (ClusterIP)  |                     |   (ClusterIP)  |
     +----------------+                     +----------------+
          |                                       |
          v                                       v
      +------------+                         +------------+
      |  Pod A1    |                         |  Pod B1    |
      +------------+                         +------------+
      |  Pod A2    |                         |  Pod B2    |
      +------------+                         +------------+
          ^                                       ^
          |                                       |
     +--------------------------------+  +--------------------------------+
     | Balanceamento de Carga pelo    |  | Balanceamento de Carga pelo    |
     | Serviço Kubernetes (ClusterIP) |  | Serviço Kubernetes (ClusterIP)|
     +--------------------------------+  +--------------------------------+
```

**Detalhes do Ingress:**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- **Controlador de Ingress**: Recebe todas as requisições externas que chegam ao cluster. Interpreta as regras definidas em **Ingress** e direciona o tráfego para os serviços corretos com base em padrões de caminho, hosts ou protocolos.
- **Regra de Ingress**: Cada regra (por exemplo, `/app1` ou `/app2`) define como o tráfego é roteado para os serviços.
- **Serviços (ClusterIP)**: Intermediários que ligam o tráfego recebido ao conjunto de **Pods** responsáveis por processar a requisição.
- **Balanceamento de Carga Interno**: O Kubernetes utiliza o balanceamento de carga entre os pods de um serviço automaticamente para distribuir o tráfego.

> **Limitações**:
>
> - Suporte a **HTTP** e **HTTPS** apenas.
> - Não há separação clara de responsabilidades (roteamento e controle de tráfego estão agrupados no Controlador de Ingress).
> - Funcionalidades avançadas, como roteamento por cabeçalhos ou manipulação de tráfego, dependem fortemente do controlador de Ingress específico.

# Representação Avançada da Gateway API

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
          +-------------------------------------------+
          | Requisições Externas (Multi-Protocolo)    |
          |       (Usuários / Aplicações)             |
          +-------------------------------------------+
                          |
                          v
           +-----------------------------------+
           |              Gateway              |
           +-----------------------------------+
                          |
        +----------------+----------------+
        |                                 |
        v                                 v
 +---------------+                 +----------------+
 |   HTTPRoute   |                 |    TCPRoute    |
 |  (para HTTP)  |                 |  (para TCP)    |
 +---------------+                 +----------------+
        |                                 |
+-----------------------+         +-----------------------+
| Correspondência de    |         | Correspondência de    |
| Caminho               |         | Porta                 |
| (/api/* -> Serviço A) |         | (Porta 3306 -> SVC DB)|
+-----------------------+         +-----------------------+
        |                                 |
        v                                 v
+----------------+                 +----------------+
|   Serviço A    |                 |   Serviço DB   |
|   (ClusterIP)  |                 |   (ClusterIP)  |
+----------------+                 +----------------+
        |                                 |
        v                                 v
  +------------+                    +------------+
  |  Pod A1    |                    |  Pod DB1   |
  +------------+                    +------------+
  |  Pod A2    |                    |  Pod DB2   |
  +------------+                    +------------+

+---------------------------------------------+
| Aplicação de Políticas (Autenticação,       |
| Limitação de Taxa, etc.)                    |
+---------------------------------------------+
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

**Detalhes da Gateway API:**

- **Gateway**: Define o ponto de entrada para o tráfego, incluindo os protocolos e portas. Um **Gateway** pode ter múltiplos **listeners** (por exemplo, para HTTP, TCP, TLS).
- **Rotas (HTTPRoute, TCPRoute, etc.)**: As rotas são definidas separadamente, permitindo uma divisão de responsabilidades. Cada rota (**HTTPRoute**, **TCPRoute**) define como o tráfego deve ser roteado e para qual serviço deve ser direcionado.
  - **HTTPRoute**: Regras específicas para tráfego HTTP, permitindo rotas baseadas em **caminhos**, **cabeçalhos** e **métodos HTTP**.
  - **TCPRoute**: Suporte a **TCP** ou outros protocolos, com roteamento baseado em portas.
- **Serviços**: Como no Ingress, os serviços são intermediários que conectam o tráfego aos pods dentro do cluster.
- **Políticas Avançadas**: A Gateway API permite aplicar políticas de tráfego diretamente nas rotas, como **autenticação**, **rate limiting** (limitação de taxa) e **TLS termination**, oferecendo um controle mais granular.

# Comparação Detalhada entre Ingress e Gateway API

| Recurso | Ingress | Gateway API |
|---|---|---|
| **Suporte a Múltiplos Protocolos** | HTTP/HTTPS apenas | Suporte a HTTP, HTTPS, TCP, TLS e mais |
| **Separação de Responsabilidades** | Regras e controle de tráfego estão agrupados | Regras de roteamento e controle de tráfego são separadas (ex: HTTPRoute, TCPRoute) |
| **Controle Granular de Tráfego** | Funcionalidades básicas, controle limitado | Controle avançado com roteamento baseado em cabeçalhos, métodos e políticas de segurança |

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

| Recurso | Ingress | Gateway API |
|---|---|---|
| **Extensibilidade** | Limitado aos controladores específicos | Suporte a extensões e integração com provedores de serviços de forma nativa |
| **Balanceamento de Carga** | Balanceamento simples entre pods via Serviço | Balanceamento mais flexível, controle via políticas diretamente no Gateway |
| **Gestão Colaborativa** | Limitada | Permite delegar controle a diferentes equipes, com gerenciamento de rotas e políticas por namespaces diferentes |
| **Suporte a Políticas** | Depende do controlador | Suporte a autenticação, rate limiting e outras políticas de rede diretamente nas rotas |

Portanto, **Ingress** e **Gateway API** são ferramentas poderosas para gerenciar o tráfego de rede em **Kubernetes**, cada uma com seus prós e contras. O **Ingress** é uma solução simples, amplamente suportada e ideal para cenários básicos de roteamento. Já a **Gateway API** oferece uma evolução significativa, permitindo uma configuração mais complexa e modular para infraestruturas avançadas. A escolha entre as duas soluções dependerá da **complexidade** do seu ambiente e das funcionalidades que você precisa implementar.

# Passo 1: Criando o Cluster com o KIND

Vamos iniciar criando um cluster Kubernetes usando o **KIND**. Para isso, precisamos de um **arquivo de configuração** que define a estrutura do nosso cluster e as portas que serão expostas.

## Criando o arquivo de configuração

Crie um arquivo chamado `kind.yaml` com o seguinte conteúdo:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 31437
        hostPort: 8080
        protocol: TCP
      - containerPort: 31438
        hostPort: 8443
        protocol: TCP
```

**Explicação:**

- **Control-plane**: Define um único nó de controle.
- **Port Mappings**: Mapeia as portas `31437` e `31438` do contêiner para `8080` e `8443` do host, respectivamente, permitindo acesso aos serviços do cluster.

## Criando o cluster

Execute o comando abaixo para criar o cluster:

```
kind create cluster --name strigus --config kind.yaml
```

- `--name strigus`: Nomeia o cluster como “strigus”.
- `--config kind.yaml`: Utiliza o arquivo de configuração que criamos.

> **Observação:** Após a execução, o **KIND** irá configurar o cluster conforme especificado. Isso pode levar alguns minutos, então tenha paciência que já já terminamos essa parte! 😉

# Passo 2: Instalando o NGINX Gateway Fabric

Com o cluster criado, o próximo passo é instalar o **NGINX Gateway Fabric**, que servirá como **controlador de tráfego** para suas aplicações.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

## Aplicando as CRDs

As **Custom Resource Definitions (CRDs)** estendem a API do Kubernetes, permitindo que o **NGINX Gateway** funcione corretamente.

```
kubectl kustomize "https://github.com/nginxinc/nginx-gateway-fabric/config/crd/gateway-api/standard?ref=v1.4.0" | kubectl apply -f -
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/v1.4.0/deploy/crds.yaml
```

**O que os comandos fazem:**

- **Kustomize**: Monta os manifestos das **CRDs** diretamente do repositório do **NGINX** no GitHub.
- **kubectl apply**: Aplica as **CRDs** ao cluster.

> **Dica:** Esse passo é essencial para que o Kubernetes reconheça os recursos específicos do **NGINX Gateway Fabric**.

## Implantando o NGINX Gateway

Agora, vamos implantar o **NGINX Gateway** utilizando o modo **NodePort**:

```
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/v1.4.0/deploy/nodeport/deploy.yaml
```

Este manifesto configura o **NGINX Gateway** para expor serviços através de **NodePort**, permitindo o acesso externo às portas configuradas.

> **Importante:** Usar o modo **NodePort** facilita o acesso aos serviços do cluster a partir do host local.

# Passo 3: Configurando o Gateway e o Serviço

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Com o **NGINX Gateway Fabric** instalado, vamos configurá-lo para gerenciar o tráfego das nossas aplicações.

## Criando o Gateway

Crie um arquivo chamado `gateway.yaml` com o seguinte conteúdo:

```yaml
apiVersion: gateway.networking.k8s.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      protocol: HTTP
      port: 80
```

**Explicação:**

- **gatewayClassName**: Define que estamos utilizando a classe de gateway do **NGINX**.
- **Listeners**: Configura um listener na porta `80` para tráfego **HTTP**.

Aplique a configuração:

```
kubectl apply -f gateway.yaml
```

> **Nota:** Este passo cria o gateway que irá gerenciar as rotas do tráfego HTTP para suas aplicações.

## Configurando o serviço do Gateway

Para expor o **NGINX Gateway**, crie o arquivo `gateway-service.yaml`:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-gateway
  namespace: nginx-gateway
  labels:
    app.kubernetes.io/name: nginx-gateway
    app.kubernetes.io/instance: nginx-gateway
    app.kubernetes.io/version: "1.4.0"
spec:
  type: NodePort
  selector:
    app.kubernetes.io/name: nginx-gateway
    app.kubernetes.io/instance: nginx-gateway
  ports:
    - name: http
      port: 80
      protocol: TCP
      targetPort: 80
      nodePort: 31437
    - name: https
      port: 443
      protocol: TCP
      targetPort: 443
      nodePort: 31438
```

**Explicação:**

- **Namespace**: O serviço está no namespace `nginx-gateway`.
- **Selector**: Garante que o serviço selecione os pods corretos.
- **Portas**: Expondo as portas `80` e `443` através de **NodePort** mapeando para `31437` e `31438`

  .

Aplique o serviço:

```bash
kubectl apply -f gateway-service.yaml
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

> **Dica:** Utilizar **NodePort** permite que você acesse o gateway através das portas mapeadas no host.

# Passo 4: Verificando a Instalação

É crucial verificar se tudo está funcionando conforme o esperado.

## Verificando os pods

Execute:

```
kubectl get pods -n nginx-gateway
```

Você deve ver algo como:

```
NAME                              READY   STATUS    RESTARTS   AGE
nginx-gateway-xxxxxxxxxx-xxxxx    1/1     Running   0          xxm
```

> **Observação:** O status `Running` indica que o pod está ativo e funcionando corretamente.

## Testando o acesso ao Gateway

Realize uma requisição `curl` para verificar a resposta do **NGINX Gateway**:

```
curl http://localhost:8080
```

Se tudo estiver correto, você receberá uma resposta do **NGINX**.

> **Dica:** Caso não receba resposta, verifique se as portas estão corretamente mapeadas e se os pods estão em execução.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Passo 5: Implantando Duas Aplicações e Testando o Roteamento

Agora, vamos implantar duas aplicações simples e configurar o **NGINX Gateway** para rotear o tráfego para elas.

## Implantando as aplicações

### Criando o Deployment da app1

Crie o arquivo `app1.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app1
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app1
  template:
    metadata:
      labels:
        app: app1
    spec:
      containers:
        - name: app1
          image: hashicorp/http-echo
          args:
            - "-text=Giropops Strigus Girus na APP1"
          ports:
            - containerPort: 5678
```

**Explicação:**

- **Replicas**: Define 2 réplicas para alta disponibilidade.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- **Container**: Utiliza a imagem `hashicorp/http-echo` que retorna uma mensagem HTTP simples.

## Criando o Deployment da app2

Crie o arquivo `app2.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app2
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app2
  template:
    metadata:
      labels:
        app: app2
    spec:
      containers:
        - name: app2
          image: hashicorp/http-echo
          args:
            - "-text=Giropops Strigus Girus na APP2"
          ports:
            - containerPort: 5678
```

**Explicação:**

- Semelhante ao deployment da **app1**, mas para a **app2**.

## Aplicando os Deployments

Execute:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
kubectl apply -f app1.yaml
kubectl apply -f app2.yaml
```

> **Dica:** Após aplicar os deployments, o Kubernetes irá criar os pods correspondentes.

### Verificando os Pods

```
kubectl get pods
```

Você deve ver algo como:

```
NAME                      READY   STATUS    RESTARTS   AGE
app1-xxxxxxxxxx-xxxxx     1/1     Running   0          xxm
app1-xxxxxxxxxx-xxxxx     1/1     Running   0          xxm
app2-xxxxxxxxxx-xxxxx     1/1     Running   0          xxm
app2-xxxxxxxxxx-xxxxx     1/1     Running   0          xxm
```

> **Observação:** Cada aplicação deve ter duas réplicas em execução.

## Criando os serviços para as aplicações

### Serviço para a app1

Crie o arquivo `svc1.yaml`:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```yaml
apiVersion: v1
kind: Service
metadata:
  name: app1-service
spec:
  selector:
    app: app1
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678
```

## Serviço para a app2

Crie o arquivo `svc2.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: app2-service
spec:
  selector:
    app: app2
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678
```

**Explicação:**

- **Selector**: Conecta o serviço aos pods com as labels correspondentes.
- **Portas**: Expondo a porta `80` para comunicação interna.

## Aplicando os Serviços

Execute:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
kubectl apply -f svc1.yaml
kubectl apply -f svc2.yaml
```

## Verificando os Serviços

```
kubectl get services
```

Você deve ver:

```
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
app1-service    ClusterIP   10.xxx.xxx.xxx  <none>        80/TCP    xxm
app2-service    ClusterIP   10.xxx.xxx.xxx  <none>        80/TCP    xxm
```

> **Nota:** Os serviços devem estar do tipo `ClusterIP`, permitindo comunicação interna dentro do cluster.

## Configurando o HTTPRoute

Vamos configurar o **HTTPRoute** para que o **NGINX Gateway** saiba para onde direcionar o tráfego com base nas URLs.

Crie o arquivo `http_route.yaml`:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```yaml
apiVersion: gateway.networking.k8s.io/v1beta1
kind: HTTPRoute
metadata:
  name: app1-app2-route
spec:
  parentRefs:
    - name: my-gateway
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: "/app1"
      backendRefs:
        - name: app1-service
          port: 80
    - matches:
        - path:
            type: PathPrefix
            value: "/app2"
      backendRefs:
        - name: app2-service
          port: 80
```

**Explicação:**

- **HTTPRoute**: Define regras de roteamento para o tráfego.
- **Regras**:
  - Requisições que começam com `/app1` são direcionadas para `app1-service`.
  - Requisições que começam com `/app2` são direcionadas para `app2-service`.

## Aplicando o HTTPRoute

Execute:

```
kubectl apply -f http_route.yaml
```

## Verificando o HTTPRoute

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
kubectl get httproute
```

Você deve ver:

```
NAME                HOSTNAMES    AGE
app1-app2-route     <none>       xxm
```

> **Dica:** O `<none>` em `HOSTNAMES` indica que o roteamento está baseado no caminho da URL.

# Testando o roteamento

## Testando a app1

Execute:

```
curl http://localhost:8080/app1
```

**Resposta Esperada:**

```
Giropops Strigus Girus na APP1
```

## Testando a app2

Execute:

```
curl http://localhost:8080/app2
```

**Resposta Esperada:**

```
Giropops Strigus Girus na APP2
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

## Analisando o Resultado

- **Roteamento**: O **NGINX Gateway** está recebendo as requisições na porta `8080` e direcionando para os serviços corretos.
- **Respostas**: Cada aplicação responde com a mensagem configurada, confirmando o roteamento adequado.

## Verificando os logs

### Logs do NGINX Gateway

Execute:

```
kubectl logs -n nginx-gateway deployment/nginx-gateway
```

Você verá logs das requisições sendo processadas pelo gateway.

### Logs das Aplicações

Para a **app1**:


1 COMENTÁRIO

Luiz Fernando disse:
20 de março de 2025 às 12:05

#VAAAI, exelente artigo

Jeff, a definição do Gateway, é um para o cluster todo, e ele vai ser responsavel por rotear para todos os HTTPRoute? ou é recomendado ter um por namespace, ou um por projeto por exemplo?

Responder

LINUXTIPS

receba nossas **novidades**, **lançamentos** e **conteúdos inéditos** gratuitamente

digite seu e-mail

ENVIAR

ao se inscrever você autoriza o envio de comunicações.

A EMPRESA

COMUNIDADE

TREINAMENTOS

INSTITUCIONAL



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

© LINUXtips 2026. todos os direitos reservados.

```
kubectl get pods -l app=app1
```

Agora, você verá **4 pods** da **app1** em execução, proporcionando maior capacidade de atendimento.

> **Nota:** O Kubernetes gerencia automaticamente o balanceamento de carga entre os pods escalados.

# Resumo da Brincadeira toda

Parabéns! \o/ Você conseguiu configurar o Gateway API no seu cluster Kubernetes com **NGINX Gateway Fabric** utilizando o **KIND**.

**Pontos-chave abordados:**

- **Configuração Inicial**: Instalando Docker, kubectl e KIND.
- **Implantação do NGINX Gateway Fabric**: Configurando CRDs e serviços.
- **Desenvolvimento e Exposição de Aplicações**: Deployments e Services.
- **Configuração de HTTPRoute**: Roteamento baseado em caminhos.

> **Dica Final:** Sempre mantenha seus componentes atualizados e monitore os logs para garantir o bom funcionamento do seu ambiente Kubernetes. Explore mais sobre **Gateway API**, **NGINX**, e práticas de **segurança** para aprimorar ainda mais suas habilidades e garantir ambientes robustos e seguros.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Recursos Adicionais

- Documentação Oficial do Kubernetes
- Documentação do KIND
- Documentação do NGINX Gateway Fabric
- Vídeos da LINUXtips sobre Kubernetes
- Cert-Manager para Gerenciamento de Certificados TLS
- OpenSSL Documentation
- Gateway API Documentation
- Ferramenta hey para Testes de Carga HTTP
- Apache JMeter

---

**Escrito por Jeferson Fernando para a LINUXtips**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF